WEEK 3

1. Number of Zeros in a Given Array

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Source Code :

```
#include <stdio.h>
int findFirstZero(int arr[], int low, int high) {
  if (low > high) {
    return -1;
  }
  int mid = low + (high - low) / 2;
  if (arr[mid] == 0) {
    if (mid == 0 || arr[mid - 1] == 1) {
      return mid;
    } else {
      return findFirstZero(arr, low, mid - 1);
    }
  } else {
    return findFirstZero(arr, mid + 1, high);
  }
}
int main() {
  int m;
  if (scanf("%d", &m) != 1) return 0;
  int arr[m];
  for (int i = 0; i < m; i++) {
    scanf("%d", &arr[i]);
  }
```

```
    int firstZeroIndex = findFirstZero(arr, 0, m - 1);
    if (firstZeroIndex == -1) {
        printf("0\n");
    } else {
        printf("%d\n", m - firstZeroIndex);
    }
    return 0;
}
```

Output :

Input	Expected	Got
5 1 1 1 0 0	2	2
10 1 1 1 1 1 1 1 1 1 1	0	0
8 0 0 0 0 0 0 0	8	8
17 1	2	2

Input	Expected	Got
1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0		

2.Majority Element

Given an array nums of size n, return *the majority element*.

The majority element is the element that appears more than ⌊n / 2⌋ times. You may assume that the majority element always exists in the array.

Example 1:

Input: nums = [3,2,3]

Output: 3

Example 2:

Input: nums = [2,2,1,1,1,2,2]

Output: 2

Constraints:

- n == nums.length
- $1 <= n <= 5 * 10^4$
- $-2^{31} <= nums[i] <= 2^{31} - 1$

For example:

Input	Result
3 3 2 3	3

Input	Result
7 2 2 1 1 1 2 2	2

Source Code :

```
#include <stdio.h>
int findMajorityElement(int arr[], int n) {
   int candidate = arr[0];
   int count = 1;
   for (int i = 1; i < n; i++) {
      if (count == 0) {
         candidate = arr[i];
         count = 1;
      } else if (arr[i] == candidate) {
         count++;
      } else {
         count--;
      }
   }
   return candidate;
}
int main() {
   int n;
   if (scanf("%d", &n) != 1) return 0;
   int arr[n];
   for (int i = 0; i < n; i++) {
      scanf("%d", &arr[i]);
   }
   int majority = findMajorityElement(arr, n);
   printf("%d\n", majority);
   return 0;
}
```

Output :

Input	Expected	Got
3 3 2 3	3	3

3.Finding Floor Value

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Source Code :

```
#include <stdio.h>
int findFloor(int arr[], int low, int high, int x) {
  int floorVal = -1;
  while (low <= high) {
    int mid = low + (high - low) / 2;
    if (arr[mid] == x) {
      return arr[mid];
    }
    if (arr[mid] < x) {
      floorVal = arr[mid];
      low = mid + 1;
    }
    else {
      high = mid - 1;
    }
  }
  return floorVal;
}
```

```
int main() {
   int n, x;
   if (scanf("%d", &n) != 1) return 0;
   int arr[n];
   for (int i = 0; i < n; i++) {
      scanf("%d", &arr[i]);
   }
   if (scanf("%d", &x) != 1) return 0;
   int result = findFloor(arr, 0, n - 1, x);
   printf("%d\n", result);
   return 0;
}
```

Output :

Input	Expected	Got
6 1 2 8 10 12 19 5	2	2
5 10 22 85 108 129 100	85	85
7 3 5 7 9 11 13 15 10	9	9

4.Two Elements sum to x

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as “No”.

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value “x”)

Source Code :

```
#include <stdio.h>
int findTwoSum(int arr[], int left, int right, int x, int *element1, int *element2) {
  if (left >= right) {
    return 0;
  }
  int currentSum = arr[left] + arr[right];
  if (currentSum == x) {
    *element1 = arr[left];
    *element2 = arr[right];
    return 1;
  }
  if (currentSum > x) {
    return findTwoSum(arr, left, right - 1, x, element1, element2);
  }
  return findTwoSum(arr, left + 1, right, x, element1, element2);
}
int main() {
  int n, x;
  if (scanf("%d", &n) != 1) return 0;
  int arr[n];
  for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
    }
    if (scanf("%d", &x) != 1) return 0;
    int element1 = 0, element2 = 0;
    if (findTwoSum(arr, 0, n - 1, x, &element1, &element2)) {
        printf("%d\n", element1);
        printf("%d\n", element2);
    } else {
        printf("No\n");
    }
    return 0;
}
```

Output :

Input	Expected	Got
4 2 4 8 10 14	4 10	4 10
5 2 4 6 8 10 100	No	No

5.Implementation of Quick Sort

Write a Program to Implement the Quick Sort Algorithm
Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.
Output:
Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Source Code :

```
#include <stdio.h>
void swap(int* a, int* b) {
   int t = *a;
   *a = *b;
   *b = t;
}
int partition(int arr[], int low, int high) {
   int pivot = arr[high];
   int i = (low - 1);
   for (int j = low; j <= high - 1; j++) {
      if (arr[j] <= pivot) {
         i++;
         swap(&arr[i], &arr[j]);
      }
   }
   swap(&arr[i + 1], &arr[high]);
   return (i + 1);
}
void quickSort(int arr[], int low, int high) {
   if (low < high) {
      int pi = partition(arr, low, high);
      quickSort(arr, low, pi - 1);
      quickSort(arr, pi + 1, high);
   }
}
int main() {
   int n;
```

```
    if (scanf("%d", &n) != 1) return 0;
    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    quickSort(arr, 0, n - 1);
    for (int i = 0; i < n; i++) {
        printf("%d", arr[i]);
        if (i < n - 1) {
            printf(" ");
        }
    }
    printf("\n");
    return 0;
}
```

Output :

Input	Expected	Got
5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98
10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114
12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90